

Lab 4, Act 3 – Answer Key

The model is wrong. Does it matter? (optional)

NRES 746 – Instructor Answer Key (not for student distribution)

This key follows the same section structure as the student handout. Questions with a single correct numeric or derived answer are answered directly; open-ended items (predictions, sketches, the plain-English procedure, reflections) are given sample answers and discussion points rather than a fixed rubric. Part 4 is a Monte Carlo experiment, so every group's numbers will land somewhere in a range, not on an exact decimal – grade on whether the direction and rough size of the numbers are right, not exact-match precision.

Part 1: What does each model assume about scatter?

1a. Writing each variance in terms of the mean μ (using each distribution's usual mean-based parameterization):

```
data.frame(  
  Distribution = c("Normal (SD sigma)", "Poisson", "Negative binomial (mean, size k)",  
                  "Gamma (mean, shape alpha)", "Lognormal (log-SD s)"),  
  Var_Y = c("sigma^2", "mean", "mean + mean^2/k", "mean^2/alpha", "(exp(s^2)-1)*mean^2"),  
  CV = c("mean^(-1) -- not constant", "mean^(-1/2) -- not constant",  
         "sqrt(1/mean+1/k) -- not constant", "1/sqrt(alpha) -- CONSTANT",  
         "sqrt(exp(s^2)-1) -- CONSTANT")  
)
```

##	Distribution	Var_Y
## 1	Normal (SD sigma)	sigma^2
## 2	Poisson	mean
## 3	Negative binomial (mean, size k)	mean + mean^2/k
## 4	Gamma (mean, shape alpha)	mean^2/alpha
## 5	Lognormal (log-SD s)	(exp(s^2)-1)*mean^2
##	CV	
## 1	mean^(-1) -- not constant	
## 2	mean^(-1/2) -- not constant	
## 3	sqrt(1/mean+1/k) -- not constant	
## 4	1/sqrt(alpha) -- CONSTANT	
## 5	sqrt(exp(s^2)-1) -- CONSTANT	

Only **gamma** and **lognormal** have a coefficient of variation (SD/mean) that doesn't depend on the mean at all – for both, the variance is a *fixed multiple* of mean², so a small mean gets a proportionally small SD and a large mean gets a proportionally large SD. Normal has constant *variance* (not CV), so its CV actually shrinks as the mean grows. Poisson and negative binomial's CV shrinks toward zero (Poisson) or toward $1/\sqrt{k}$ (negative binomial) as the mean grows, but neither is exactly constant.

Common mistake to watch for: confusing “constant variance” with “constant CV” – they're opposite behaviors. Normal is the textbook example of constant variance and is the one distribution in this table with *non-constant* CV.

1b.

```

s <- sigma
medians <- c(60, 45, 6)
means <- medians * exp(s^2 / 2)
sds <- means * sqrt(exp(s^2) - 1)
data.frame(
  row = c("Year 0", "Year 25, mild", "Year 25, severe"),
  median = medians, mean = round(means, 2), sd = round(sds, 2)
)

```

```

##           row median  mean   sd
## 1      Year 0      60 61.21 12.37
## 2  Year 25, mild      45 45.91  9.27
## 3 Year 25, severe       6  6.12  1.24

```

Two things worth flagging: the **mean is always a bit above the median** (about 2% higher here, since $e^{s^2/2} = e^{0.02} \approx 1.0202$ regardless of which row) – lognormal distributions are right-skewed, so a few large draws pull the mean up past the middle value. And the **SD shrinks in lockstep with the mean** ($SD/mean = \sqrt{e^{0.04} - 1} \approx 0.2013$ in all three rows) – this is the constant-CV property from 1a made concrete: as the population declines, the scatter around it shrinks proportionally rather than staying a fixed number of animals.

1c. No single correct sketch, but a correct one shows a curved (not straight) declining median, a $\pm 2SD$ band that narrows as the median narrows, and a straight regression line cutting through both. The clearest logical flaw shows up right at the end of the severe scenario: a line fit through 25 points that start at $I_0 = 60$ and end near a median of 6 has to have a fairly steep negative slope, and extended slope means the fitted line **goes negative before year 25**:

```

med_severe <- I0 * exp(b_severe * t)
m_check <- lm(med_severe ~ t)
c(fitted_t0 = predict(m_check, data.frame(t = 0)), fitted_t25 = predict(m_check, data.frame(t = 25)))

## fitted_t0.1 fitted_t25.1
## 47.0871203 -0.4102682

```

A fitted line predicting **negative animals** is not just an approximation error – it’s a logically impossible prediction, and it’s a giveaway that a straight line is the wrong shape for a process that can only decay toward (never through) zero. The mild scenario’s line, by contrast, stays positive across the whole range: the flaw is there in principle but much less visually obvious, which is itself worth pointing out to a group that didn’t predict a difference between the two scenarios here.

Part 2: Commit to predictions

2a. The **log-scale analyst is unbiased in both scenarios**, because it’s the model that matches how the data were actually generated – nothing about the size of the decline changes that. The **raw-scale analyst’s bias depends on the scenario**: over a *mild* decline the exponential curve is close enough to a straight line across 25 years that a straight-line fit is a reasonable local approximation, so the bias is small. Over a *severe* decline the curve bends sharply and a straight line cannot follow it – the raw-scale analyst systematically **overestimates the percent decline** (predicts a bigger loss than actually happened), because fitting a straight line through a convex-decreasing curve undershoots the high early values and can run past zero before year 25.

```

sim_analysts <- function(b, R, seed) {
  set.seed(seed)
  pct_log <- numeric(R); pct_raw <- numeric(R)
  for (i in seq_len(R)) {
    idx <- I0 * exp(b * t + rnorm(length(t), 0, sigma))
    m_log <- lm(log(idx) ~ t)
  }
}

```

```

pct_log[i] <- 1 - exp(25 * unname(coef(m_log)[2]))
m_raw <- lm(idx ~ t)
fit0 <- unname(coef(m_raw)[1]); fit25 <- unname(coef(m_raw)[1] + coef(m_raw)[2] * 25)
pct_raw[i] <- 1 - fit25 / fit0
}
c(mean_log = mean(pct_log), mean_raw = mean(pct_raw))
}
rbind(mild = sim_analysts(b_mild, 2000, 401), severe = sim_analysts(b_severe, 2000, 402))

##           mean_log mean_raw
## mild    0.2400219 0.2398107
## severe  0.8994658 1.0079530

```

This confirms it numerically: mean estimated decline is about 24% (mild, both analysts close to the true 25%) versus about 90% log-scale / **101% raw-scale** (severe) – an average estimate of *more than total* loss, which foreshadows the “share > 100%” column students will fill in during Part 4.

Common misconception to watch for: assuming any misspecified model must be biased, in every scenario, by a similar amount – the mild scenario shows a wrong model can still give approximately correct answers when the wrong assumption doesn’t matter much locally. The size of the model error, not just its presence, determines the size of the bias.

Part 3: A goodness of fit check

3a. A complete plain-English procedure should include, in some form:

1. Fit the model (log-scale or raw-scale, whichever is being checked) to the real data; extract the fitted values and the residual SD.
2. Count the sign changes in the real residuals – call this s_{real} .
3. Repeat many times (e.g., 200):
 - a. Simulate one new dataset the same length as the real data, using the *fitted* values as the mean and the *fitted* residual SD as the noise (not the unknown true b and σ – those aren’t available in real life).
 - b. Refit the *same* model to this simulated dataset and count its sign changes.
4. The p -value is the fraction of the simulated datasets with sign changes $\leq s_{\text{real}}$.

Common omissions to watch for: simulating from the *true* generating model instead of the *fitted* model – the whole point of a goodness-of-fit check is that in real life you never know the truth, only the fit; forgetting to refit the model to each simulated dataset before counting its sign changes (the check needs the same fitting procedure applied consistently, not just noise added to the original fit); being vague about what “extreme” means (a two-sided phrasing like “as different from expected” is fine, but the handout’s own wording – “as few sign changes as the real data” – is one-sided and should be followed to keep the exercise well-defined).

3b. No single correct wording, but the direction should track the same intuition as 2a: a model can only be caught by this check if its errors are systematic (all runs the same direction over a stretch of years), not just larger in magnitude.

```

##           model  mild  severe
## 1 Log-scale model rarely  rarely
## 2 Raw-scale model rarely usually

```

The log-scale model is correctly specified in both scenarios, so it should get flagged only at about its nominal false-positive rate (around 5% of the time, by chance) regardless of how severe the decline is. The raw-scale model is flagged **usually** only in the severe scenario, where the curvature is severe enough to leave a systematic run of same-signed residuals; in the mild scenario its errors are small and patternless enough that this particular check usually doesn’t catch it, even though the model is still technically wrong.

Checkpoint: finalize your plan

No fixed answer – the point is for each student to catch, on their own, anything underspecified in their 3a procedure (most commonly: simulating from the true model instead of the fitted model, or not refitting before recounting sign changes) before writing any code.

Part 4: Run the experiments

```
sign_changes <- function(r) {
  s <- sign(r)
  sum(s[-1] != s[-length(s)])
}

plot_envelope <- function(t, y, sims) {
  lo <- apply(sims, 1, quantile, 0.05)
  hi <- apply(sims, 1, quantile, 0.95)
  plot(t, y, type = "n", ylim = range(y, lo, hi), xlab = "Year", ylab = "Count index")
  polygon(c(t, rev(t)), c(lo, rev(hi)), col = "grey85", border = NA)
  points(t, y, pch = 16)
  abline(h = 0, lty = 3)
}
```

Experiment 1: two analysts

```
run_exp1 <- function(b, R, seed) {
  set.seed(seed)
  pct_log <- numeric(R); pct_raw <- numeric(R)
  for (i in seq_len(R)) {
    idx <- I0 * exp(b * t + rnorm(length(t), 0, sigma))
    m_log <- lm(log(idx) ~ t)
    pct_log[i] <- 1 - exp(25 * unname(coef(m_log)[2]))
    m_raw <- lm(idx ~ t)
    fit0 <- unname(coef(m_raw)[1]); fit25 <- unname(coef(m_raw)[1] + coef(m_raw)[2] * 25)
    pct_raw[i] <- 1 - fit25 / fit0
  }
  list(pct_log = pct_log, pct_raw = pct_raw)
}

mild <- run_exp1(b_mild, 2000, 401)
severe <- run_exp1(b_severe, 2000, 402)
summarize <- function(x) c(mean = mean(x) * 100, sd = sd(x) * 100, share_over_100 = mean(x > 1))
res1 <- rbind(
  mild_log = summarize(mild$pct_log), mild_raw = summarize(mild$pct_raw),
  severe_log = summarize(severe$pct_log), severe_raw = summarize(severe$pct_raw)
)
round(res1, 3)
```

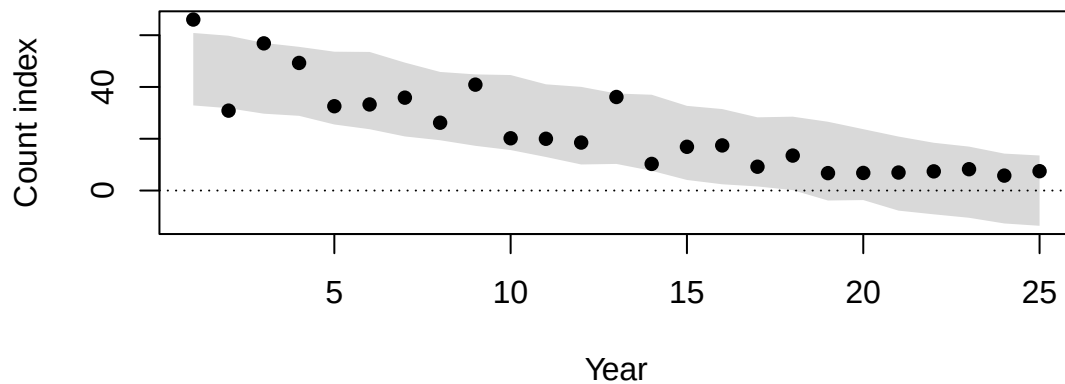
##	mean	sd	share_over_100
## mild_log	24.002	10.850	0.000
## mild_raw	23.981	10.972	0.000
## severe_log	89.947	1.411	0.000
## severe_raw	100.795	2.929	0.608

At the handout's $R = 2,000$, expect something close to: **mild** scenario, both analysts land near the true 25% decline (mean $\approx 24\%$) with similar spread (SD ≈ 11 points), and essentially nobody predicts more than 100% loss. **Severe** scenario is where the two analysts split: log-scale is tight around the true 90% (SD ≈ 1.4 points), while raw-scale averages **just past 100%** decline with much more spread (SD ≈ 2.9 points) and a

large share of replicates (roughly 50-65%) predicting **more than a total loss** – a nonsensical answer that should be an immediate red flag to a working analyst, model-checking aside.

Experiment 2: an envelope

```
set.seed(4703)
idx_severe <- I0 * exp(b_severe * t + rnorm(length(t), 0, sigma))
m_raw <- lm(idx_severe ~ t)
sims <- replicate(500, fitted(m_raw) + rnorm(length(t), 0, summary(m_raw)$sigma))
plot_envelope(t, idx_severe, sims)
```



What the envelope shows: in the severe scenario, the raw-scale model's own predictive envelope dips **below zero** for the later years – the model is confident enough in its (wrong) straight-line shape that it predicts a real chance of a negative count index, which is impossible. A student doesn't need a formal test to distrust a model whose own simulated replicate datasets include impossible values; this is the same idea as Part 3's check, made visual instead of numeric. (In the mild scenario, run the same code with `b_mild` – the envelope stays comfortably positive throughout, another way the two scenarios diverge.)

Experiment 3: the goodness-of-fit check

```
gof_check <- function(y, n_inner = 200) {
  m <- lm(y ~ t)
  sc_real <- sign_changes(residuals(m))
  fitted_vals <- fitted(m); resid_sd <- summary(m)$sigma
  sc_sim <- replicate(n_inner, {
    y_sim <- fitted_vals + rnorm(length(t), 0, resid_sd)
    sign_changes(residuals(lm(y_sim ~ t)))
  })
  mean(sc_sim <= sc_real)
}
run_exp3 <- function(b, n_outer = 100, seed) {
  set.seed(seed)
  p_raw <- numeric(n_outer); p_log <- numeric(n_outer)
  for (i in seq_len(n_outer)) {
    idx <- I0 * exp(b * t + rnorm(length(t), 0, sigma))
```

```

    p_raw[i] <- gof_check(idx)
    p_log[i] <- gof_check(log(idx))
  }
  c(log_flag_rate = mean(p_log < 0.05), raw_flag_rate = mean(p_raw < 0.05))
}
res3 <- rbind(mild = run_exp3(b_mild, 100, 501), severe = run_exp3(b_severe, 100, 502))
res3

##          log_flag_rate raw_flag_rate
## mild              0.03           0.05
## severe            0.01           0.68

```

The **log-scale model** gets flagged at roughly its nominal false-positive rate (somewhere around 0-10% with only 100 outer replicates – this is Monte Carlo noise around 5%, not a problem) in **both** scenarios, since it's correctly specified either way. The **raw-scale model** is flagged rarely in the mild scenario but **usually** (well over half the time, typically 60-75%) in the severe scenario – exactly the pattern predicted in 3b, and it's the one cell in this table that should look clearly different from the rest.

Common bugs to watch for (especially AI-generated code):

- Simulating the inner check's fake datasets from the *true* b/σ instead of the *fitted* model – this silently turns the goodness-of-fit check into something closer to the Part 4 Experiment 1 check, and will make even the raw-scale model look fine in the mild scenario, since it compares against the wrong reference distribution.
- Forgetting to refit `lm()` inside the inner loop before calling `sign_changes()` on the simulated data – reusing the *real* data's fitted model's residuals for every simulated dataset makes `sc_sim` constant and the check meaningless.
- Applying `log()` to the raw-scale model's residuals or fitted values by mistake instead of taking `log(idx)` *before* fitting – these look similar but fit two different lines.
- Not using `sign()`'s off-by-one indexing correctly in `sign_changes()` (`s[-1] != s[-length(s)]` compares each residual to its immediate predecessor); a common bug is comparing every residual to the *first* one instead, which will produce implausibly small sign-change counts.

If you want to go even further (I_0 and Jensen's inequality)

```

set.seed(99)
i0_sim <- I0 * exp(rnorm(100000, 0, sigma))
c(mean_log = mean(log(i0_sim)),
  log_mean = log(mean(i0_sim)),
  gap = log(mean(i0_sim)) - mean(log(i0_sim)),
  sigma_sq_over_2 = sigma^2 / 2)

```

```

##          mean_log      log_mean          gap sigma_sq_over_2
##      4.09310326      4.11303040      0.01992714      0.02000000

```

The gap matches $\sigma^2/2$ almost exactly, as the closed-form lognormal mean formula from 1b predicts (mean = $e^{m+\sigma^2/2}$, so $\log(\text{mean}) = m + \sigma^2/2$, while $\text{mean}(\log(I_0)) = m$ on average). The average of the logs is *smaller* than the log of the average because $\log(\cdot)$ is a concave function: **Jensen's inequality** says $E[\log(X)] \leq \log(E[X])$ for any random X , with equality only if X has no spread at all. This is worth connecting back to 1b's finding that the lognormal mean sits above the median – both are the same underlying fact (right skew pulls the arithmetic mean up) showing up two different ways.

Reflection

No fixed answer. A good reflection should name the specific scenario (severe, not mild) where the wrong model actually mattered in a way that would change a management decision, and should connect that back

to a concrete signal from Part 4 – the raw-scale model’s bias, its wider spread, its physically impossible predictions in the envelope, or its high goodness-of-fit flag rate – rather than a generic “the wrong model was wrong” statement. Worth watching for: some students may initially predict in 2a/3b that *any* model misspecification should be easy to detect, and be surprised that the mild scenario’s raw-scale model passes both the eyeball test and the formal check almost as often as the correct model – a real result, not a bug, and a useful caution against assuming every wrong model announces itself.